



Department of Decision Science

Faculty of Business

University of Moratuwa

Semester 04

DA2111

Statistical and Machine Learning

Group Assignment

Due Date of Submission

[24/11/2024]



Department of Decision Sciences

Faculty of Business

University of Moratuwa

Semester 04

DA2111-Statistical and Machine Learning

Group Formation Sheet

	NAME	INDEX NUMBER
01	Fernando W.M.C.H.	226034C
02	Gayasha K.J.P.	226038R
03	Kumara K.P.H.U.	226066B
04	Thilakarathna I.I.S.	226120L

Table of Contents

1. Data Set Selection	3
1.1 Dataset Overview	3
1.2 Dataset Structure	3
1.3 Column Descriptions	4
2. Problem Definition.....	6
2.1 Target Variable	6
2.2 Business Problem.....	6
3. Data Exploration and Preprocessing	7
3.1 Exploratory Steps.....	7
3.1.1 Summarize the structure and characteristics of the data set	7
3.1.2 Identify variables, their types, and relationships between them	8
3.2 Preprocessing Steps	9
3.2.1 Dropping columns.....	9
3.2.1 Finding and removing Duplicates.....	9
3.2.1 Handle Missing Values	10
3.2.2 Encode Categorical Variables.....	12
3.2.3 Scale Numerical Features	13
4. Exploratory Data Analysis (EDA).....	15
4.1 Identify outliers or anomalies in the data.....	15
4.2 Visualize the distribution of key features and spot patterns or trends	17
4.3 Assess correlations between variables to understand interdependencies	19
4.3.1 Heatmaps and Pair plots were drawn to explore correlations.....	19
4.3.2 Pivot tables or group-based aggregation for summarizing trends in categorical data	22
5. Model Selection	24
5.1. Logistic Regression.....	24
5.2 Gradient Boosting Classifier.....	25
5.3 Random Forest	26
6. Model Training and Evaluation	27
6.1 Initial Analysis of Class Distribution.....	27
6.2 Class Balancing.....	28

6.3 Data splitting	29
6.4 Train each selected model on the training set	29
6.5 Evaluate the performance of each model using appropriate metrics	30
7. Hyperparameter Tuning	33
7.1 Fine-tune the hyperparameters of the model to improve performance	33
7.1.1 Logistic Regression	33
7.1.2 Random Forest	34
7.1.3 Gradient Boosting Classifier	35
7.2 Compare model performance before and after Tuning	36
8. Comparison and Conclusion	38
8.1 Comparison	38
8.2 Challenges encountered during the project and Solutions	40
8.3 Conclusion Remarks	41
9. Documentation and Presentation	42

1. Data Set Selection

Data Set Download link: <https://www.kaggle.com/datasets/wenruiiu/adult-income-dataset/data>

1.1 Dataset Overview

The selected dataset addresses a meaningful socio-economic problem: predicting income levels based on demographic, educational, and employment-related features. The dataset contains anonymized records of individuals and can be used for classification tasks to identify whether a person's annual income exceeds \$50,000. This analysis has practical applications in economic policy-making, targeted marketing, and resource allocation.

1.2 Dataset Structure

```
# 1. Summarize the structure and characteristics of the adult dataset
```

```
print(df.shape)
print(df.info())
```

```
[ ] import numpy as np
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
```

```
(48842, 15)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48842 entries, 0 to 48841
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   48842 non-null  int64
1   workclass             48842 non-null  object
2   fnlwgt                48842 non-null  int64
3   education             48842 non-null  object
4   educational-num       48842 non-null  int64
5   marital-status        48842 non-null  object
6   occupation            48842 non-null  object
7   relationship          48842 non-null  object
8   race                  48842 non-null  object
9   gender                48842 non-null  object
10  capital-gain          48842 non-null  int64
11  capital-loss          48842 non-null  int64
12  hours-per-week        48842 non-null  int64
13  native-country        48842 non-null  object
14  income                48842 non-null  object
dtypes: int64(6), object(9)
memory usage: 5.6+ MB
None
```

Shape: 48,842 rows and 15 columns

Categorical Features (9): workclass, education, marital-status, occupation, relationship, race, gender, native-country, income

Numerical Features (6): age, fnlwgt, educational-num, capital-gain, capital-loss, hours-per-week

1.3 Column Descriptions

Column Name	Description	Type	Notes
age	Age of the individual	Numerical	Continuous variable; generally expected to influence income level.
workclass	Employment type or sector (e.g., Private, Self-Employed)	Categorical (Nominal)	Reflects socio-economic background; missing values denoted as ?.
fnlwgt	Final weight: A census variable representing population count	Numerical	Indicates sampling relevance; not directly meaningful for prediction.
education	Highest education level achieved (e.g., Bachelors, HS-grad)	Categorical (Ordinal)	Reflects educational qualifications.
education-num	Numerical encoding of education levels	Numerical	Derived from education; treated as ordinal.
marital-status	Marital status (e.g., Married-civ-spouse, Never-married)	Categorical (Nominal)	May indicate income stability or economic behavior.
occupation	Job type or industry (e.g., Tech-support, Craft-repair)	Categorical (Nominal)	Missing values denoted as ?; indicative of earning potential and economic activity.
relationship	Family relationship (e.g., Husband, Wife, Unmarried)	Categorical (Nominal)	Describes social relationships; overlaps with marital status.
race	Race of the individual (e.g., White, Black, Asian-Pac-Islander)	Categorical (Nominal)	Sensitive feature; use cautiously to avoid bias in predictions.
sex	Gender of the individual (Male or Female)	Categorical (Nominal)	Affects income trends and workplace participation.
capital-gain	Capital gains from investments	Numerical	Highly skewed; most entries are 0. Outliers may need special treatment.
capital-loss	Capital losses from investments	Numerical	Highly skewed; most entries are 0. Outliers may need special treatment.
hours-per-week	Hours worked per week	Numerical	Continuous variable; significant predictor of income potential.

native-country	Country of origin (e.g., United-States, Mexico)	Categorical (Nominal)	Missing values denoted as ?; may affect employment opportunities and income.
income	Target variable: Income group ($\leq 50K$ or $> 50K$)	Categorical (Binary)	Binary classification target to predict income level.

Justification for Selection

This dataset is ideal because it,

- Addresses a real-world problem with direct applications in multiple fields.
- Includes a mix of categorical and numerical variables for diverse feature engineering.
- Is large enough (48,842 records) to ensure robust statistical analysis and machine learning model training.
- Contains a well-defined binary target variable, making it suitable for classification tasks.

2. Problem Definition

2.1 Target Variable

The target variable in the selected dataset is **income**, which is a binary categorical variable:

Values

- **<= 50K**: Individuals earning \$50,000 or less annually
- **> 50K**: Individuals earning more than \$50,000 annually

This variable will be used to classify individuals into two income groups based on their demographic, educational, and employment-related features.

2.2 Business Problem

Can we accurately predict whether an individual earns more than \$50,000 annually based on their demographic, educational, and employment characteristics?

This problem is highly relevant in areas like:

- **Economic Policy**: Understanding income disparities based on demographic and socio-economic factors.
- **Targeted Marketing**: Identifying high-income individuals for tailored advertising and product recommendations.
- **Workforce Planning**: Analyzing employment trends and labor market segmentation.

3. Data Exploration and Preprocessing

3.1 Exploratory Steps

3.1.1 Summarize the structure and characteristics of the data set

The dataset contains **48,842 rows** and **15 columns** with a mix of numerical and categorical features. A detailed summary of the dataset is as follows.

```
[ ] # 1. Summarize the structure and characteristics of the adult dataset

print(df.shape)
print(df.info())
```

```
(48842, 15)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48842 entries, 0 to 48841
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                    48842 non-null  int64
1   workclass              48842 non-null  object
2   fnlwgt                 48842 non-null  int64
3   education              48842 non-null  object
4   educational-num        48842 non-null  int64
5   marital-status         48842 non-null  object
6   occupation             48842 non-null  object
7   relationship           48842 non-null  object
8   race                   48842 non-null  object
9   gender                 48842 non-null  object
10  capital-gain           48842 non-null  int64
11  capital-loss           48842 non-null  int64
12  hours-per-week         48842 non-null  int64
13  native-country         48842 non-null  object
14  income                 48842 non-null  object
dtypes: int64(6), object(9)
memory usage: 5.6+ MB
None
```

Here we calculate some Summary Statistics for Numerical Features.

```
[ ] df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	48842.0	38.643585	13.710510	17.0	28.0	37.0	48.0	90.0
fnlwgt	48842.0	189664.134597	105604.025423	12285.0	117550.5	178144.5	237642.0	1490400.0
educational-num	48842.0	10.078089	2.570973	1.0	9.0	10.0	12.0	16.0
capital-gain	48842.0	1079.067626	7452.019058	0.0	0.0	0.0	0.0	99999.0
capital-loss	48842.0	87.502314	403.004552	0.0	0.0	0.0	0.0	4356.0
hours-per-week	48842.0	40.422382	12.391444	1.0	40.0	40.0	45.0	99.0

3.1.2 Identify variables, their types, and relationships between them

```
# 2. Identify variables and their types
numerical_features = df.select_dtypes(include='number').columns.tolist()
categorical_features = df.select_dtypes(include='object').columns.tolist()
df.describe(include="object").T
```

	count	unique	top	freq
workclass	48842	9	Private	33906
education	48842	16	HS-grad	15784
marital-status	48842	7	Married-civ-spouse	22379
occupation	48842	15	Prof-specialty	6172
relationship	48842	6	Husband	19716
race	48842	5	White	41762
gender	48842	2	Male	32650
native-country	48842	42	United-States	43832
income	48842	2	<=50K	37155

Categorical Features: 9

- *workclass, education, marital-status, occupation, relationship, race, gender, native-country, income*

Numerical Features: 6

- *age, fnlwgt, educational-num, capital-gain, capital-loss, hours-per-week*

Most frequent values in **Categorical Features**:

- **workclass**: Private (33,906)
- **education**: HS-grad (15,784)
- **marital-status**: Married-civ-spouse (22,379)
- **native-country**: United-States (43,832)

Potential relationships:

- **Capital-gain** and **capital-loss** are likely to influence **income**.
- **Hours-per-week** positively correlates with income levels.
- Demographic factors such as **age**, **education**, and **occupation** are significant predictors.

3.2 Preprocessing Steps

3.2.1 Dropping columns

Dropping the fnlwgt column

The fnlwgt column was removed as it does not provide meaningful information relevant to the analysis. This column represents the final weight assigned to individuals during data collection and does not contribute to the understanding or prediction of the target variable (income).

```
[ ] # Drop the 'fnlwgt' column from the DataFrame as it doesnot give a meaningful interpretation.  
df = df.drop(columns=['fnlwgt'])
```

Dropping the education column

The education column was also removed because it is redundant. The dataset includes another numerical column, educational-num, which conveys the same information as education in a more precise manner (representing years of education). Keeping both columns could lead to multicollinearity, which may negatively impact model performance. Therefore, to avoid redundancy and ensure better data representation, only educational-num was retained.

```
[ ] # Drop the 'education' column from the DataFrame as it doesnot give new information to the model.  
# This column is same as "educational_num" column  
df = df.drop(columns=['education'])
```

3.2.1 Finding and removing Duplicates

Using the df.duplicated() method, 6,374 duplicate records were identified in the dataset. However, a detailed inspection revealed that none of the rows were entirely identical across all attributes. These duplicates occurred due to shared values in certain columns (e.g., individuals with the same age and occupation), which is expected and does not necessarily indicate true redundancy.

Given that these records represent meaningful overlaps rather than true duplicates, no rows were removed from the dataset, ensuring the integrity of the data for analysis.

```
df.duplicated().sum()  
6374
```

3.2.1 Handle Missing Values

```
# Handle Missing Values:
count = df.isnull().sum()
missing_df = pd.DataFrame({"count": count})
missing_df
```

	count
age	0
workclass	0
educational-num	0
marital-status	0
occupation	0
relationship	0
race	0
gender	0
capital-gain	0
capital-loss	0
hours-per-week	0
native-country	0
income	0

In the dataset, there were no explicitly missing or empty values detected using the standard null-checking code (`df.isnull().sum()`).

However, upon closer inspection, certain entries in columns such as `workclass`, `occupation`, and `native-country` contained `"?"`, which effectively represent missing values. Therefore, we will have to handle them considering them as missing values.

```
df.isin(["?"]).sum()
# It can be seen that a "?" is included in some entries for column workclass, occupation and native country, therefore we will have to handle them considering them as missing values.
```

	0
age	0
workclass	2799
educational-num	0
marital-status	0
occupation	2809
relationship	0
race	0
gender	0
capital-gain	0
capital-loss	0
hours-per-week	0
native-country	857
income	0

dtype: int64

1. Mode Imputation

```
[ ] # Imputation
df.workclass = df.workclass.replace("?", "Private")
# The entries with "?" in the workclass were replaced with "Private" because it is the most frequent value (mode imputation) in the workclass data.

[ ] df['native-country'] = df['native-country'].replace("?", "United-States")
# The same imputation method was applied here as "United States" was the most frequent value.
```

For Workclass and Native-Country columns, the **mode** (most frequent value) of each column was used to replace the missing entries:

- For workclass, the mode was "Private", which was used to fill in the missing values.
- For native-country, the mode was "United States", which replaced the missing values.

2. Forward fill Technique

```
[ ] df.replace({'occupation': '?'}, value=np.nan, inplace=True)
df['occupation'] = df['occupation'].ffill()

# Here "?" in the occupation column was replaced using the "forward fill" technique.
```

The "?" entries in the occupation column were imputed using the **forward-fill (ffill)** method. This technique propagates the last valid value forward, ensuring that the imputation maintains a logical continuity within the data.

These methods were chosen to preserve the integrity of the dataset and ensure minimal distortion of its statistical properties.

3.2.2 Encode Categorical Variables

In preparing the adult dataset for classification analysis, the categorical variables were encoded using the OneHotEncoder from the sklearn.preprocessing module. This encoding transforms nominal categorical variables, such as workclass, marital-status, and gender, into new binary columns, each representing one of the unique categories. By setting drop='first', the first category for each variable was dropped to avoid the dummy variable trap. These new columns were then added to the dataset to replace the original categorical columns.

```
# Identify nominal columns, excluding 'native-country' for manual processing
nominal_cols = ['workclass', 'marital-status', 'occupation', 'relationship', 'race', 'gender', 'income']

# Create the column 'native-country-US'
df['native-country-US'] = (df['native-country'] == 'United-States').astype(float)

# Create the column 'native-country-Others'
df['native-country-Others'] = 1 - df['native-country-US']

# Encode remaining nominal columns using One-Hot Encoding
ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded_nominal = pd.DataFrame(ohe.fit_transform(df[nominal_cols]), columns=ohe.get_feature_names_out(nominal_cols))

# Concatenate the encoded nominal columns with the original DataFrame, dropping the processed nominal columns
df = pd.concat([df.drop(columns=nominal_cols), encoded_nominal], axis=1)

# Drop 'native-country' if it's still there
if 'native-country' in df.columns:
    df.drop(columns=['native-country'], inplace=True)

# Display the first few rows to confirm changes
print(df.head())
```

```
age  educational-num  capital-gain  capital-loss  hours-per-week  \
0    25              7            0            0             40
1    38              9            0            0             50
2    28             12            0            0             40
3    44             10           7688            0             40
4    18             10            0            0             30

native-country-US  native-country-Others  workclass_Local-gov  \
0                1.0                  0.0                  0.0
1                1.0                  0.0                  0.0
2                1.0                  0.0                  1.0
3                1.0                  0.0                  0.0
4                1.0                  0.0                  0.0

workclass_Never-worked  workclass_Private  ...  \
0                    0.0                1.0  ...
1                    0.0                1.0  ...
2                    0.0                0.0  ...
3                    0.0                1.0  ...
4                    0.0                1.0  ...

relationship_Other-relative  relationship_Own-child  \
0                        0.0                      1.0
1                        0.0                      0.0
2                        0.0                      0.0
3                        0.0                      0.0
4                        0.0                      1.0

relationship_Unmarried  relationship_Wife  race_Asian-Pac-Islander  \
0                    0.0                0.0                  0.0
1                    0.0                0.0                  0.0
2                    0.0                0.0                  0.0
3                    0.0                0.0                  0.0
4                    0.0                0.0                  0.0

race_Black  race_Other  race_White  gender_Male  income_>50K
0         1.0        0.0        0.0         1.0         0.0
1         0.0        0.0        1.0         1.0         0.0
2         0.0        0.0        1.0         1.0         1.0
3         1.0        0.0        0.0         1.0         1.0
4         0.0        0.0        1.0         0.0         0.0

[5 rows x 44 columns]
```

Additionally, 2 new binary columns were created to replace the original native-country column.

- **native-country-US:**

Indicates whether the individual's country is the United States (1 for "United States," 0 otherwise).

- **native-country-Others:**

The complement of native-country-US (1 for all other countries, 0 for "United States").

This was done because the majority of the data was collected from US nationals, making "United-States" a prominent category. Transforming the native-country variable into these two columns helped simplify the data representation for the regression model while preserving essential distinctions.

The transformation step ensures that all features are now numerical, with proper binary encoding for categorical variables.

3.2.3 Scale Numerical Features

```
[ ] # Define the constant C for log transformation
    C = 1 # You can adjust this based on your dataset and scaling needs

    # Initialize the StandardScaler
    scaler = StandardScaler()

    # Apply log transformation and scaling to all numeric columns in the original DataFrame
    for col in df.select_dtypes(include=['int64']).columns:
        # Log transformation with constant C
        df[col] = np.log1p(df[col] + C) # log1p(x + C)
        # Apply Standard Scaler to log-transformed data
        df[col] = scaler.fit_transform(df[[col]])

    # Exclude one-hot encoded columns (e.g., 'income_>50K' and others)
    exclude_columns = df.filter(regex='income_>50K|native-country|workclass|marital-status|occupation|relationship|race|gender').columns

    # Get the columns to plot (excluding the one-hot encoded columns and 'income_>50K')
    columns_to_plot = df.select_dtypes(include=['float64', 'int64']).drop(columns=exclude_columns)

    # Visualize the scaled data (box plots for numerical columns)
    plt.figure(figsize=(18, 12))
    for i, col in enumerate(columns_to_plot.columns):
        plt.subplot(6, 3, i + 1)
        sns.boxplot(data=df, y=col, color='orange')
        plt.title(f'Scaled {col}')

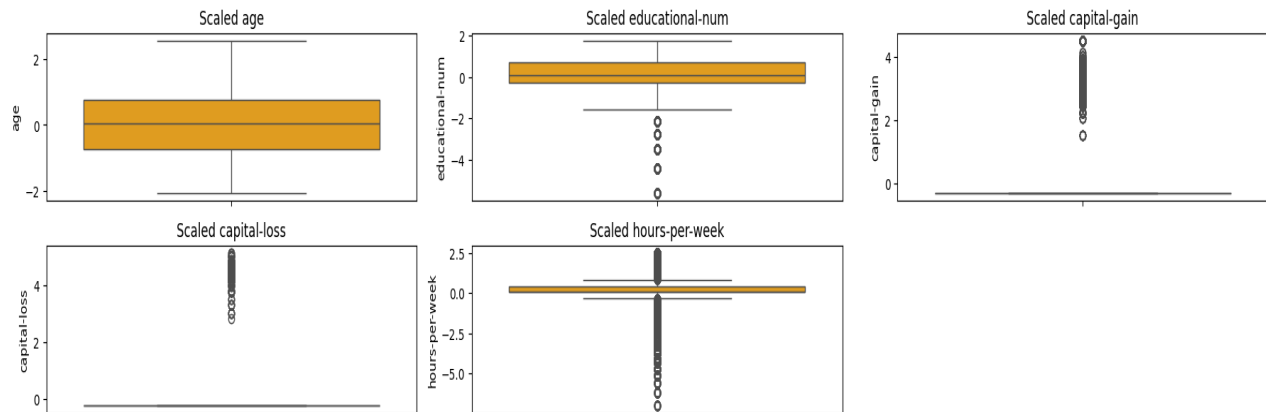
    plt.tight_layout()
    plt.show()
```

To ensure a robust preprocessing pipeline for our adult income dataset, we focused on addressing skewed distributions and differences in scales among features. Specifically, two variables—**capital gain** and **capital loss**—exhibited highly skewed distributions, with the majority of values being zeros. These zeros represent individuals not engaged in stock market activities, as we assume capital gains and losses were recorded through stock market transactions. The presence of zeros is natural in this context, but the few non-zero values were detected as outliers when applying conventional outlier detection techniques. Removing or capping these values would have falsified the dataset, compressing all data into the zero range and leading to misleading conclusions.

To handle this challenge, we applied a **log transformation with a constant C=1**. This transformation mitigated the skewness in the data, ensuring that all non-zero values were retained while reducing the long tails of the distribution. By using the log transformation, we preserved the

importance of non-zero values as they are critical for analyzing income levels and distinguishing individuals who engage in stock market activities.

After the log transformation, we applied **StandardScaler** to standardize the features, ensuring they have a mean of 0 and a standard deviation of 1. Standard scaling is crucial for making features comparable, as variables like *capital gain* and *capital loss* differ greatly in their ranges compared to others, such as *educational_num* or *age*. This preprocessing ensured that all features contributed equally to the classification model, ultimately improving its performance in predicting the target variable.



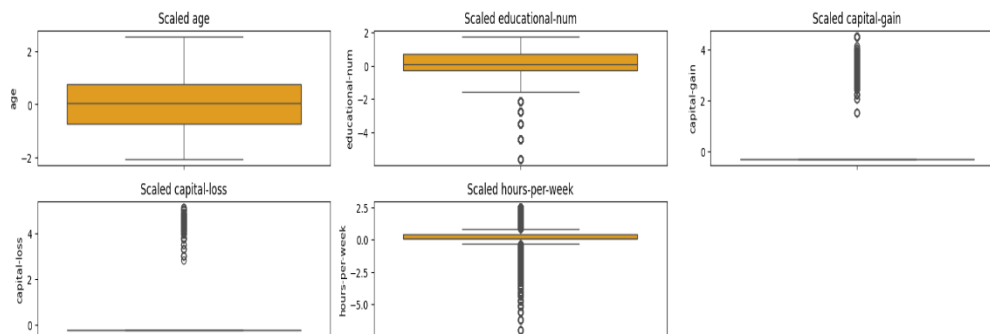
4. Exploratory Data Analysis (EDA)

In this analysis, we imported several key libraries to perform Exploratory Data Analysis (EDA) on the wine dataset:

- **Pandas** (pd): For data manipulation and analysis, facilitating the handling of tabular data.
- **NumPy** (np): For numerical computing, providing support for arrays and mathematical functions.
- **Matplotlib** (plt): For creating various plots and visualizations to represent data distributions and relationships.
- **Seaborn** (sns): For enhanced statistical graphics, simplifying the creation of informative visualizations.

Together, these libraries enable comprehensive exploration and analysis of the dataset.

4.1 Identify outliers or anomalies in the data



To handle the outliers in the dataset, we applied the **clipping method** to specific variables, including *age*, *educational-num*, and *hours-per-week*. These variables were identified as having outliers even after applying log transformations and standard scaling.

```
# List of columns to apply clipping, including 'educational-num'
columns_to_clip = ['age', 'hours-per-week', 'educational-num'] # Add 'educational-num' here

# Define the lower and upper percentiles for clipping (e.g., 5th and 95th percentile)
lower_percentile = 0.05
upper_percentile = 0.95

# Apply clipping to selected columns in the original DataFrame
for col in columns_to_clip:
    lower_limit = df[col].quantile(lower_percentile)
    upper_limit = df[col].quantile(upper_percentile)

    # Clip the values to the specified percentiles
    df[col] = df[col].clip(lower=lower_limit, upper=upper_limit)

# Visualize the original and clipped data using boxplots to compare
plt.figure(figsize=(14, 10))

# Boxplots after clipping (Clipped Data)
for i, col in enumerate(columns_to_clip):
    plt.subplot(2, len(columns_to_clip), len(columns_to_clip) + i + 1)
    sns.boxplot(data=df, y=col, color='green')
    plt.title(f'Clipped {col}')

plt.tight_layout()
plt.show()
```

When considering the 2 variables *capital gain* and *capital loss* in the dataset, most values are zero, while the non-zero values fall within a realistic range of 1,000 to 99,999. These variables represent income-related aspects, making it unrealistic for their values to fall below 10. Despite their skewed distribution, these features are crucial for assessing an individual's income category and cannot be dismissed as outliers. Treating all non-zero values as outliers and applying a clipping method would result in most values being clipped to zero, which would misrepresent the data and lead to inaccurate conclusions. Instead, we addressed the skewness using log transformation, which preserves the non-zero values while reducing their influence, ensuring that the variables remain meaningful for predicting income categories.

Therefore, clipping method was used only for *age*, *educational-num*, and *hours-per-week* since they are not as critical for making income-related predictions. Clipping ensures that extreme outliers in these variables are constrained to the 5th and 95th percentiles, thus reducing their impact on model performance without distorting the dataset.

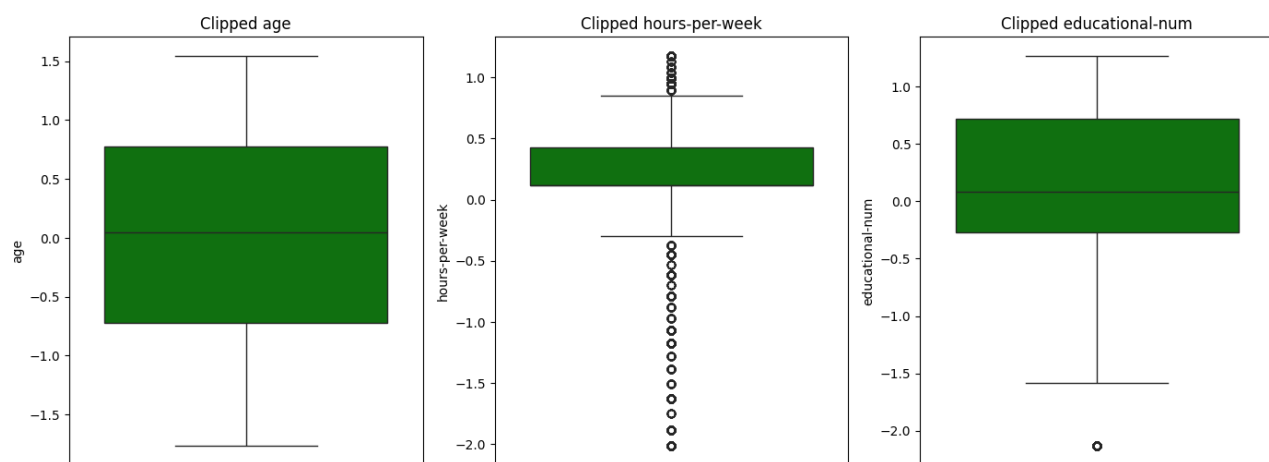
Here's a summary of the approach:

- **Clipping Method for Outliers:**

- We clipped values in *age*, *educational-num*, and *hours-per-week* to the 5th and 95th percentiles, ensuring that extreme outliers did not adversely affect the results.
- This approach was chosen specifically for these variables because they are less sensitive to extreme values compared to *capital gain* and *capital loss*, which required more careful treatment.

- **Boxplots for Visualization:**

- We visualized both the original and clipped distributions of these variables using boxplots to compare how outliers were handled. The clipped distributions show a more controlled spread of values, ensuring that extreme outliers no longer skew the analysis.



4.2 Visualize the distribution of key features and spot patterns or trends

After detecting and handling the outliers the visualizations of the key features of the distributions were done as follows,

Histograms for data distributions:

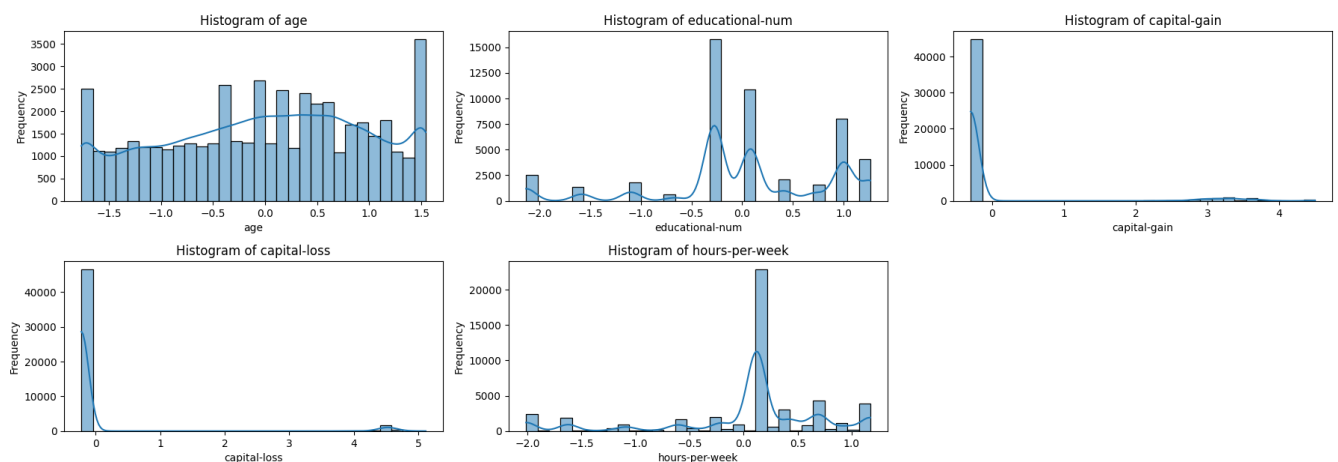
```
# Exclude one-hot encoded columns and 'income >50K' column from the scaled dataset
exclude_columns = df.filter(regex='income_>50K|native-country|workclass|marital-status|occupation|relationship|race|gender').columns

# Get the columns to plot (excluding the specified columns)
columns_to_plot = df.select_dtypes(include=['float64', 'int64']).drop(columns=exclude_columns)

# Set up the plotting area with enough room for histograms
plt.figure(figsize=(18, 12)) # Adjust figure size for better spacing

# Loop through each feature and draw a histogram
for i, column in enumerate(columns_to_plot.columns, 1):
    plt.subplot(4, 3, i) # Create a 4x3 grid for 12 features (adjust rows/columns as needed)
    sns.histplot(data=df, x=column, bins=30, kde=True) # Adding kde=True for kernel density estimate
    plt.title(f'Histogram of {column}')
    plt.xlabel(column)
    plt.ylabel('Frequency')

plt.tight_layout() # Adjust spacing
plt.show()
```



Histogram of Age

This shows the distribution of the age feature after normalization or scaling (likely StandardScaler). The data seems fairly spread out, with some peaks around certain intervals. This suggests age groups that are more common in the dataset.

Histogram of Educational-Num

The distribution of educational-num (possibly an encoded or scaled numeric representation of education levels) shows distinct peaks. These peaks likely correspond to specific educational attainment levels that are more common in the dataset, suggesting categorical-like data.

Histogram of Capital-Gain

The data for capital-gain appears highly skewed, even after handling outliers, with most values clustered near zero. This is typical for income-related datasets where only a small subset of people reports significant capital gains.

Histogram of Capital-Loss

Similar to capital-gain, capital-loss is also highly skewed, with most values concentrated at or near zero. This reflects the rarity of reported capital losses.

Histogram of Hours-per-Week

The distribution of hours-per-week is concentrated near certain intervals, suggesting common working hours. Peaks around the center suggest standard full-time or part-time working hours.

4.3 Assess correlations between variables to understand interdependencies

4.3.1 Heatmaps and Pair plots were drawn to explore correlations

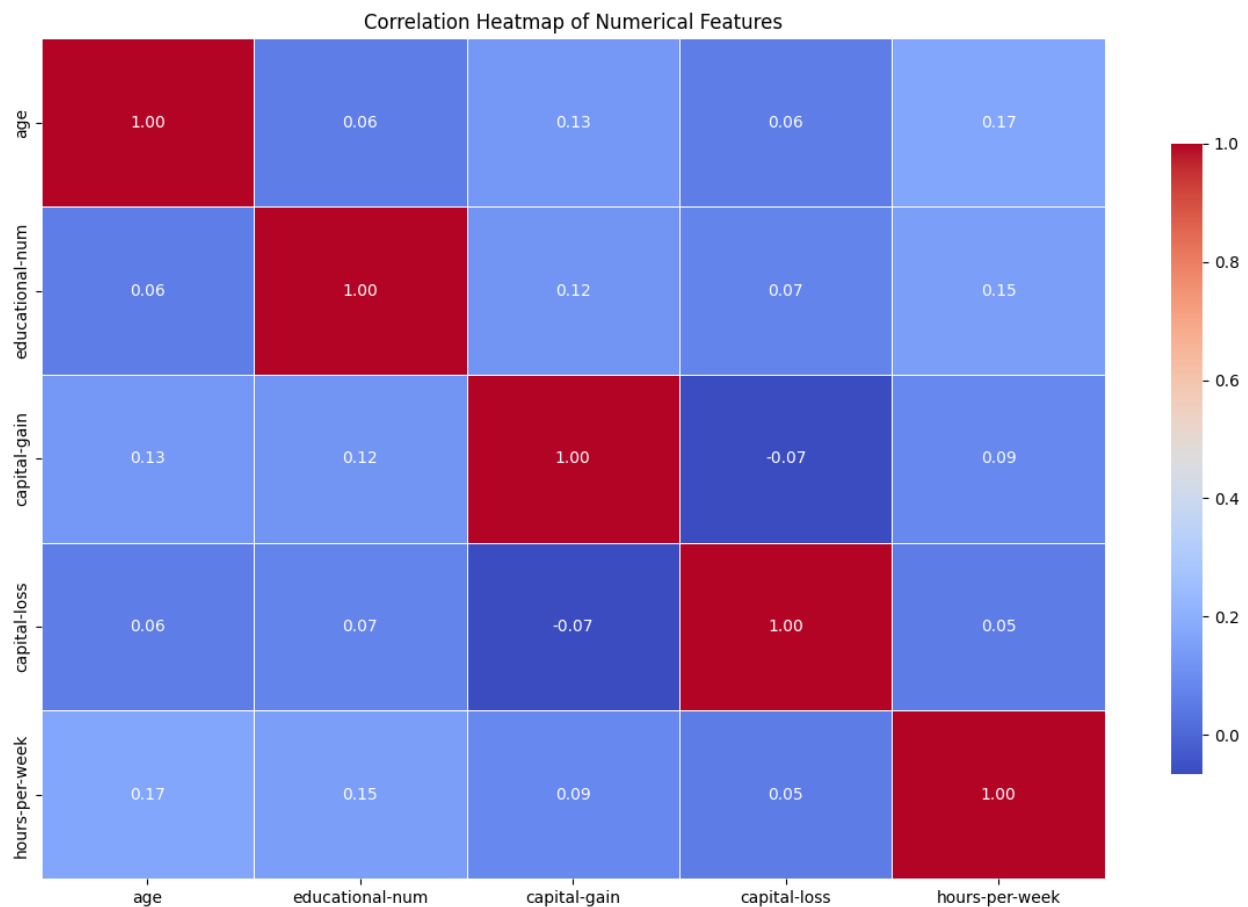
Heatmaps

```
# Exclude unwanted columns explicitly
exclude_columns = df.filter(regex='income_>50K|native-country|workclass|marital-status|occupation|relationship|race|gender').columns

# Select strictly numerical columns for the heatmap
columns_to_plot = df.select_dtypes(include=['float64', 'int64']).drop(columns=exclude_columns).columns

# Compute the correlation matrix for selected columns
corr_matrix = df[columns_to_plot].corr()

# Plot the heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', linewidths=0.5, cbar_kws={"shrink": 0.75})
plt.title("Correlation Heatmap of Numerical Features")
plt.tight_layout()
plt.show()
```



The heatmap visualizes the pairwise Pearson correlation coefficients between the selected numerical features in the dataset. Correlation values range from:

- **-1**: Perfect negative correlation (as one variable increases, the other decreases).
- **0**: No linear correlation.
- **+1**: Perfect positive correlation (as one variable increases, the other also increases).

Observations from the Heatmap

- **age vs. capital-gain**: Weak positive correlation (0.13). Older individuals tend to have slightly higher capital gains.
- **educational-num vs. hours-per-week**: Weak positive correlation (0.12). People with higher education levels tend to work slightly longer hours.
- **capital-gain vs. capital-loss**: Weak negative correlation (-0.07). There is no significant linear relationship between these features.

Features like hours-per-week and capital-gain show minimal correlation with others, indicating no multicollinearity issues that might affect the machine learning model.

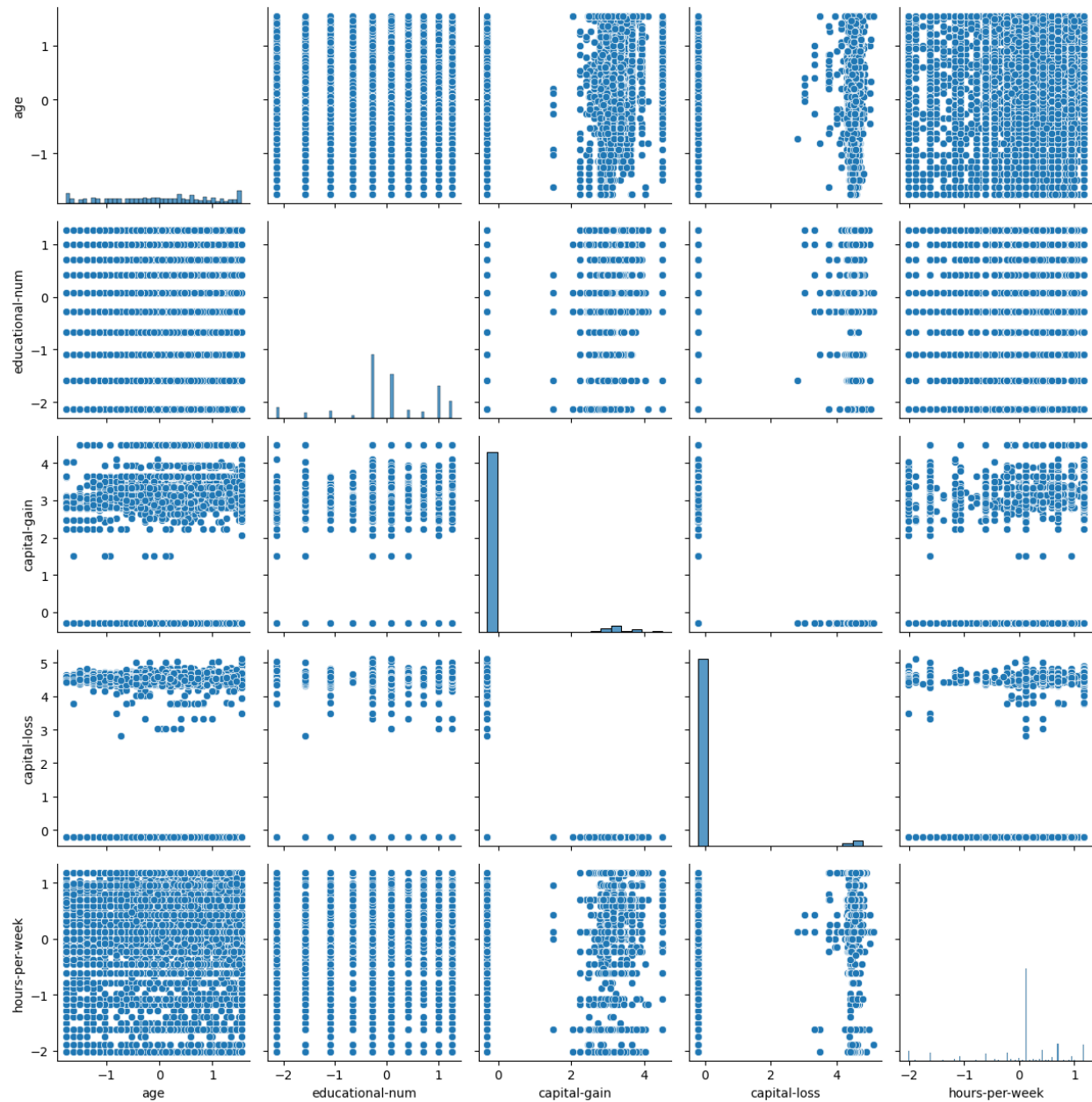
Pair plots

```
# Exclude one-hot encoded columns and 'income >50K' column from the scaled dataset
exclude_columns = df.filter(regex='income_>50K|native-country|workclass|marital-status|occupation|relationship|race|gender').columns

# Get the columns to plot (excluding the specified columns)
columns_to_plot = df.select_dtypes(include=['float64', 'int64']).drop(columns=exclude_columns)

# Set up the pairplot for the selected numeric columns
sns.pairplot(columns_to_plot)

# Adjust layout for better spacing
plt.tight_layout()
plt.show()
```



· Variable Relationships

- Some variables like age, capital-gain, and capital-loss show high concentration in specific ranges.
- capital-gain and capital-loss have many repeated zero values, suggesting a significant portion of the data is zero.

· Skewed Distributions

- Variables such as capital-gain, capital-loss, and hours-per-week exhibit outliers or long tails.

· Potential Correlations

- No strong linear correlations are visible, though age and hours-per-week show some patterns.

4.3.2 Pivot tables or group-based aggregation for summarizing trends in categorical data

```
# List of categorical variables that are one-hot encoded
categorical_variables = ['workclass', 'marital-status', 'occupation', 'relationship', 'race', 'gender', 'native-country']

# Initialize an empty list to store results
all_grouped_results = []

# Loop through each categorical variable
for var in categorical_variables:
    # Get the one-hot encoded columns for the current categorical variable
    var_columns = [col for col in df.columns if f'{var}_' in col]

    # Loop through each one-hot encoded column to compute aggregations
    for col in var_columns:
        # Filter rows where the column value is 1 (i.e., category present)
        temp_df = df[df[col] == 1]

        # Perform aggregation
        aggregation = {
            'variable': var, # Keep track of the categorical variable
            'category': col, # Keep track of the category (column name)
            'average_age': temp_df['age'].mean(),
            'average_hours_per_week': temp_df['hours-per-week'].mean(),
            'average_educational_num': temp_df['educational-num'].mean(),
            'count': temp_df['age'].size
        }

        all_grouped_results.append(aggregation)

# Convert the results into a DataFrame
grouped_df = pd.DataFrame(all_grouped_results)

# Display the result
print("Grouped by One-Hot Encoded Categorical Variables with Aggregation:")
print(grouped_df)
```

Grouped by One-Hot Encoded Categorical Variables with Aggregation:			
	variable	category	average_age \
0	workclass	workclass_Local-gov	0.244538
1	workclass	workclass_Never-worked	-1.563575
2	workclass	workclass_Private	-0.121024
3	workclass	workclass_Self-emp-inc	0.508462
4	workclass	workclass_Self-emp-not-inc	0.460059
5	workclass	workclass_State-gov	0.088962
6	workclass	workclass_Without-pay	0.374105
7	marital-status	marital-status_Married-AF-spouse	-0.484052
8	marital-status	marital-status_Married-civ-spouse	0.368612
9	marital-status	marital-status_Married-spouse-absent	0.156713
10	marital-status	marital-status_Never-married	-0.825330
11	marital-status	marital-status_Separated	0.142402
12	marital-status	marital-status_Widowed	1.179272
13	occupation	occupation_Armed-Forces	-0.331112
14	occupation	occupation_Craft-repair	0.060247
15	occupation	occupation_Exec-managerial	0.267536
16	occupation	occupation_Farming-fishing	0.128489
17	occupation	occupation_Handlers-cleaners	-0.442517
18	occupation	occupation_Machine-op-inspct	-0.054110
19	occupation	occupation_Other-service	-0.301815
20	occupation	occupation_Priv-house-serv	0.150460
21	occupation	occupation_Prof-specialty	0.155474
22	occupation	occupation_Protective-serv	0.041382
23	occupation	occupation_Sales	-0.114992
24	occupation	occupation_Tech-support	-0.075836
25	occupation	occupation_Transport-moving	0.146785
26	relationship	relationship_Not-in-family	-0.032662
27	relationship	relationship_Other-relative	-0.444727
28	relationship	relationship_Own-child	-1.124464
29	relationship	relationship_Unmarried	0.163809
30	relationship	relationship_Wife	0.158634
31	race	race_Asian-Pac-Islander	-0.046703
32	race	race_Black	-0.042046
33	race	race_Other	-0.357150
34	race	race_White	0.001731
35	gender	gender_Male	0.063686

	average_hours_per_week	average_educational_num	count
0	0.089603	0.343961	3136
1	-0.655756	-0.097841	10
2	0.000391	-0.041946	36705
3	0.458102	0.353643	1695
4	0.186249	0.060509	3862
5	-0.038728	0.429495	1981
6	-0.473927	-0.306279	21
7	-0.072308	0.157070	37
8	0.223521	0.099485	22379
9	-0.008242	-0.187469	628
10	-0.195684	0.007785	16117
11	0.021328	-0.231095	1530
12	-0.431045	-0.292925	1518
13	-0.069715	0.284575	17
14	0.155381	-0.270825	6510
15	0.286515	0.450309	6459
16	0.247334	-0.445979	1570
17	-0.130862	-0.465538	2223
18	0.085580	-0.456036	3226
19	-0.365466	-0.382492	5202
20	-0.512026	-0.741043	261
21	0.112692	0.005508	6557
22	0.141764	0.072604	1031
23	-0.000111	0.115448	5813
24	-0.010160	0.328974	1531
25	0.226242	-0.391749	2507
26	0.059301	0.121408	12583
27	-0.161797	-0.359415	1506
28	-0.467681	-0.147168	7581
29	-0.011109	-0.104752	5125
30	-0.184505	0.172383	2331
31	0.005063	0.324778	1519
32	-0.050530	-0.152348	4685
33	-0.017327	-0.367947	406
34	0.051436	0.050903	41762
35	0.166354	0.034658	32650

The >50K group demonstrates a pattern of higher age, longer work hours, and greater financial activity (both gains and losses).

The <=50K group has younger individuals with fewer working hours and minimal involvement in capital transactions.

5. Model Selection

Problem Type: Classification

The goal is to predict whether an individual's income is $\leq 50K$ or $> 50K$, which is a binary classification problem. Based on this, we choose 3 algorithms

- Logistic Regression
- Random forest
- Gradient Boosting Classifier

5.1. Logistic Regression

Logistic Regression is a linear model that estimates the probability of an outcome using the logistic (sigmoid) function. It works well for binary classification when the relationship between features and the target variable is approximately linear.

Advantages

- It's simple, interpretable, and computationally efficient.
- Works well if the dataset features are linearly separable or close to linear.
- Provides probabilistic outputs, which can be useful for threshold-based decision-making.

Justification

- ✓ **Nature of the Problem:** Logistic Regression is a simple and interpretable algorithm for binary classification problems. Since the target variable (income $\leq 50K$ or $> 50K$) is binary, this algorithm is well-suited for the task.
- ✓ **Linear Relationships:** If the features are linearly related to the target variable (or can be transformed to exhibit such relationships), Logistic Regression can perform well.
- ✓ **Interpretability:** Coefficients of the logistic regression model provide insights into how each feature impacts the prediction, which is important in understanding factors influencing income.
- ✓ **Efficiency:** Logistic Regression is computationally efficient and works well with moderately sized datasets, making it a good baseline model.
- ✓ **Data Assumptions:** It assumes no multicollinearity and linear relationships between features and the log-odds, so preprocessing steps like scaling and encoding categorical data are essential.

5.2 Gradient Boosting Classifier

Gradient Boosting is another ensemble method that builds trees sequentially, where each tree corrects the errors of the previous ones. It optimizes a loss function and often delivers high accuracy on structured data.

Advantages

- Highly effective at capturing complex patterns and interactions in the data.
- Allows for regularization to prevent overfitting.
- Outperforms Random Forest in many cases due to its iterative approach in reducing model error.
- Works well on imbalanced datasets if tuned properly (e.g., by adjusting class weights).

Justification

- ✓ **High Predictive Power:** Gradient Boosting models generally outperform other algorithms in classification problems because they optimize for model accuracy by sequentially reducing errors in prediction.
- ✓ **Nonlinear Relationships and Complex Patterns:** Like Random Forest, Gradient Boosting captures nonlinear relationships and interactions between features, making it powerful for datasets with intricate patterns.
- ✓ **Hyperparameter Flexibility:** Gradient Boosting models provide a high degree of flexibility through tunable hyperparameters, enabling optimization for maximum performance.
- ✓ **Handling Imbalanced Data:** With appropriate hyperparameters (e.g., class weights), Gradient Boosting can handle imbalanced datasets effectively, which is often a concern in income prediction.
- ✓ **Feature Importance:** Similar to Random Forest, Gradient Boosting provides feature importance, helping with feature selection and interpretation.
- ✓ **Efficiency and Scalability:** Implementations like XGBoost and LightGBM are highly optimized and can handle large datasets efficiently.

5.3 Random Forest

Random Forest is an ensemble method that builds multiple decision trees using different subsets of the data and features. It aggregates their predictions for robust and accurate classification.

Advantages

- Handles non-linear relationships between features and the target variable very well.
- Robust to outliers and less prone to overfitting than individual decision trees.
- Automatically handles interactions between features without requiring explicit feature engineering.
- Provides feature importance scores, which can aid in feature selection.

Justification

- ✓ **Nonlinear Relationships:** Random Forest can capture complex, nonlinear relationships between the features and the target variable, making it robust for datasets where relationships are not easily modeled by linear techniques.
- ✓ **Handling Feature Interactions:** It automatically detects interactions between features, which is particularly useful if certain combinations of features (e.g., age and education level) significantly influence income.
- ✓ **Outlier and Noise Robustness:** Random Forest is robust to outliers and noise in the data, which is important for real-world datasets like this one.
- ✓ **Feature Importance:** It provides feature importance scores, helping identify the most impactful predictors of income.
- ✓ **Scalability:** It can handle large datasets effectively and is less sensitive to overfitting than a single decision tree, due to averaging over multiple trees.
- ✓ **Missing Data Handling:** Random Forest can handle missing values, reducing the need for imputation in preprocessing.

Comparison of Models

Model	Strengths	Weaknesses
Logistic Regression	Simple, interpretable, computationally efficient, and works well on linear data.	Struggles with non-linear relationships; requires scaled data and feature preprocessing.
Random Forest	Captures non-linear patterns, robust to outliers, handles mixed data types, and provides feature importance.	May overfit on noisy data; can be computationally expensive for large datasets.
Gradient Boosting Classifier	High accuracy, excellent handling of non-linear relationships, and effective in reducing bias.	Slower training compared to Random Forest; sensitive to hyperparameters and prone to overfitting.

6. Model Training and Evaluation

6.1 Initial Analysis of Class Distribution

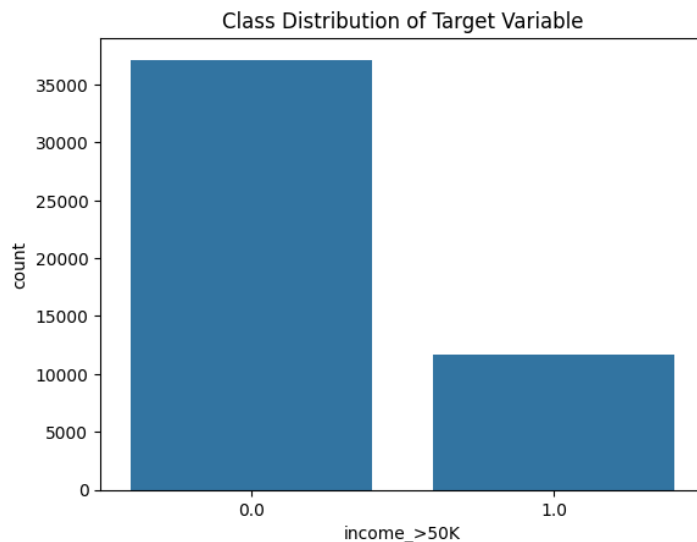
```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, roc_auc_score
```

```
# Define the target variable and features
target_variable=['income_>50K']
features = [col for col in df.columns if col not in target_variable]

print("Target Variable:", target_variable)
print("Features:", features)

# Check the class distribution
print(df["income_>50K"].value_counts())

# Visualize class distribution
sns.countplot(x='income_>50K', data=df)
plt.title('Class Distribution of Target Variable')
```



The target variable income_>50K is **highly imbalanced**.

- **Class 0 (<=50K):** 37,155 samples.
- **Class 1 (>50K):** 11,687 samples.

This means the dataset has approximately **76% of samples in Class 0** and only **24% in Class 1**. Imbalanced datasets are problematic for machine learning algorithms because they tend to **favor the majority class** (Class 0 in this case) and perform poorly on the minority class (Class 1).

Balancing the dataset ensures that the model has equal exposure to both classes. This helps,

1. **Avoid bias towards the majority class.**
2. **Improve performance metrics** like recall, precision, and F1-score, particularly for the minority class.
3. **Train a more robust model** capable of handling both classes effectively.

6.2 Class Balancing

```
[ ]
x = df[features].to_numpy() # 2D array (features)
y = df['income_>50K'].to_numpy()

# Apply SMOTE to balance the classes
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

# Check the new shape of the resampled data
print("Original dataset shape:", pd.Series(y).value_counts())
print("Resampled dataset shape:", pd.Series(y_resampled).value_counts())
```

```
Original dataset shape: 0.0    37155
1.0    11687
Name: count, dtype: int64
Resampled dataset shape: 0.0    37155
1.0    37155
Name: count, dtype: int64
```

The number of samples in the minority class (Class 1) was **increased to match the majority class (Class 0)**. After resampling, the new dataset shape is,

- **Class 0 (<=50K):** 37,155 samples.
- **Class 1 (>50K):** 37,155 samples.

Both classes now have equal representation, achieving a **balanced dataset** using SMOTE oversampling method.

6.3 Data splitting

To ensure we can evaluate the model's performance on unseen data, we first split the data into training and test sets.

```
[ ] # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2, random_state=42)
```

`train_test_split` is used to divide the dataset into training and test sets, with 80% of the data used for training and 20% reserved for testing.

6.4 Train each selected model on the training set

```
[ ] from sklearn.ensemble import RandomForestClassifier
    from sklearn.ensemble import GradientBoostingClassifier

    log_reg = LogisticRegression()
    rf_classifier = RandomForestClassifier(random_state=42)
    gb_classifier = GradientBoostingClassifier(random_state=42)

    # Train the models on the training set
    log_reg.fit(X_train, y_train)
    rf_classifier.fit(X_train, y_train)
    gb_classifier.fit(X_train, y_train)
```

▼ GradientBoostingClassifier ⓘ ?

GradientBoostingClassifier(random_state=42)

6.5 Evaluate the performance of each model using appropriate metrics

Now, we will evaluate each model using metrics such as accuracy, precision, recall, F1-score, and confusion matrix. For demonstration, we'll use `classification_report` and `confusion_matrix` for evaluation.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix
)

# Function to evaluate classification models
def evaluate_classification_model(model, X_train, y_train, X_test, y_test):
    # Predictions on training data
    y_train_pred = model.predict(X_train)
    y_train_prob = model.predict_proba(X_train)[:, 1]

    # Predictions on test data
    y_test_pred = model.predict(X_test)
    y_test_prob = model.predict_proba(X_test)[:, 1]

    # Training set metrics
    train_metrics = {
        "Accuracy": accuracy_score(y_train, y_train_pred),
        "Precision": precision_score(y_train, y_train_pred),
        "Recall": recall_score(y_train, y_train_pred),
        "F1-score": f1_score(y_train, y_train_pred),
        "AUC-ROC": roc_auc_score(y_train, y_train_prob),
    }

    # Testing set metrics
    test_metrics = {
        "Accuracy": accuracy_score(y_test, y_test_pred),
        "Precision": precision_score(y_test, y_test_pred),
        "Recall": recall_score(y_test, y_test_pred),
        "F1-score": f1_score(y_test, y_test_pred),
        "AUC-ROC": roc_auc_score(y_test, y_test_prob),
    }

    # Print evaluation metrics
    print(f"Model: {model.__class__.__name__}")
    print("")

    print("Training Metrics:")
    for metric, value in train_metrics.items():
        print(f"{metric}: {value:.4f}")
    print("\nTesting Metrics:")
    for metric, value in test_metrics.items():
        print(f"{metric}: {value:.4f}")
    print("")

    # Confusion Matrix for Test Data
    cm = confusion_matrix(y_test, y_test_pred)
    labels = [str(i) for i in sorted(set(y_test))] # Dynamically set labels
    plt.figure(figsize=(6, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels, yticklabels=labels)
    plt.title(f'Confusion Matrix - {model.__class__.__name__}')
    plt.ylabel('True Labels')
    plt.xlabel('Predicted Labels')
    plt.show()
    print("")

    return {"train_metrics": train_metrics, "test_metrics": test_metrics}

# Evaluate each model
log_reg_metrics = evaluate_classification_model(log_reg, X_train, y_train, X_test, y_test)
rf_classifier_metrics = evaluate_classification_model(rf_classifier, X_train, y_train, X_test, y_test)
gb_classifier_metrics = evaluate_classification_model(gb_classifier, X_train, y_train, X_test, y_test)
```

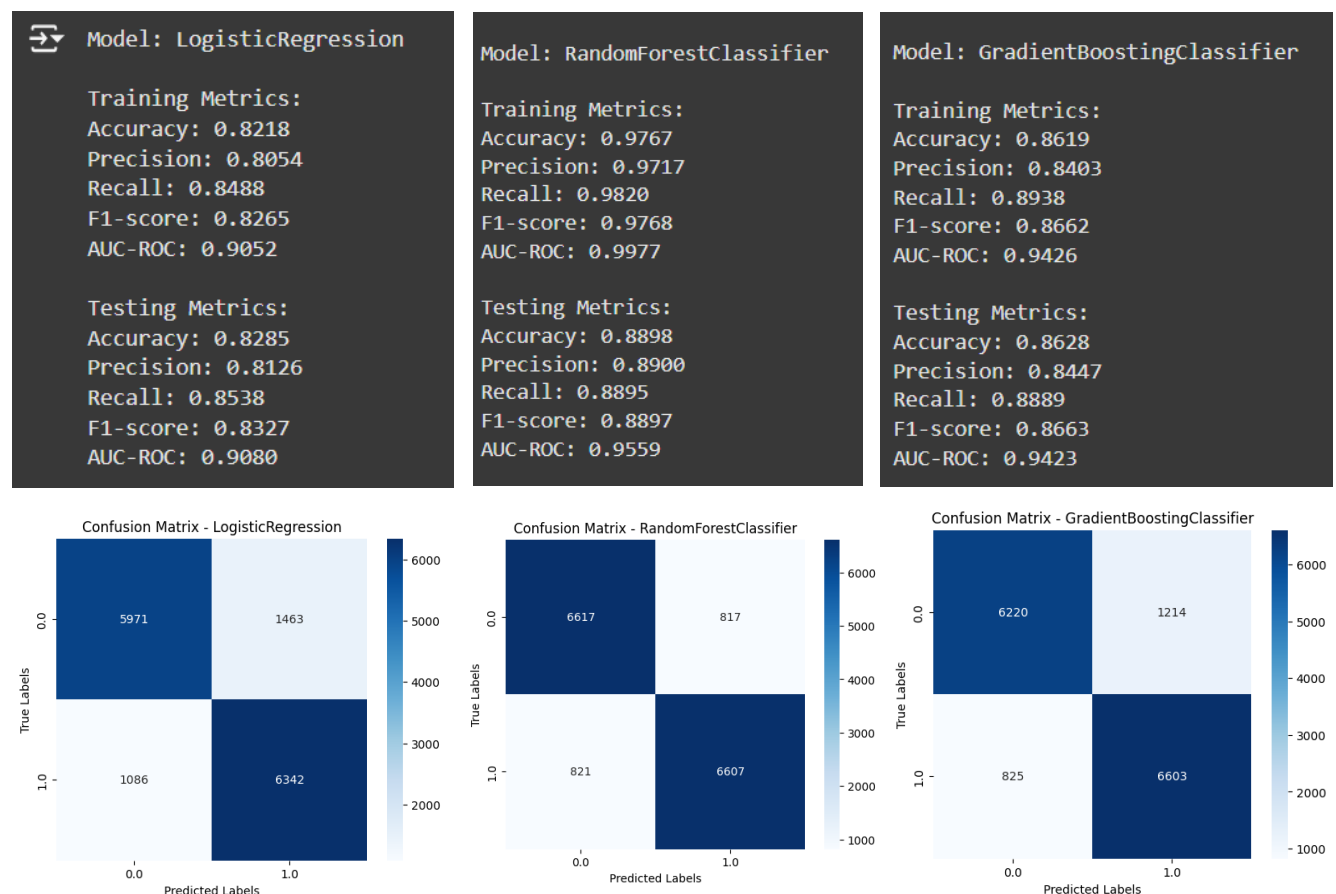
Accuracy: Measures the proportion of correct predictions (both positive and negative) out of all predictions. However, accuracy can be misleading if the dataset is imbalanced.

Precision: Represents the proportion of true positive predictions out of all positive predictions made by the model. High precision indicates fewer false positives.

Recall (Sensitivity): Represents the proportion of true positives identified out of all actual positive samples. High recall indicates fewer false negatives.

F1-Score: The harmonic mean of precision and recall, balancing both metrics. Useful when precision and recall are equally important.

AUC-ROC (Area Under the Receiver Operating Characteristic Curve): Measures the ability of the model to distinguish between classes. Higher values (close to 1) indicate better performance.



- Logistic Regression performs adequately, but its performance is slightly weaker compared to the ensemble models, especially for recall (sensitivity).

- The Random Forest model performs significantly better, especially on testing data, with a good balance of precision and recall. However, its high training accuracy suggests slight overfitting.
- Gradient Boosting strikes a good balance between training and testing metrics, avoiding significant overfitting. It has slightly lower overall performance than Random Forest but is more stable.

Based on the overall performance, Random Forest Classifier provides the highest accuracy (97.67%) and maintain a good balance between precision and recall across all classes. Both handle class imbalances well and exhibit fewer misclassifications compared to Logistic Regression and Gradient Boosting Classifier.

7. Hyperparameter Tuning

7.1 Fine-tune the hyperparameters of the model to improve performance

We'll start by tuning the hyperparameters of each model using Grid Search or Randomized Search to improve model performance.

1. Logistic Regression
2. Random Forest
3. Hyper Parameter Turing

7.1.1 Logistic Regression

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.linear_model import LogisticRegression

# Hyperparameter distributions for Logistic Regression
param_distributions_log_reg = {
    'C': [0.01, 0.1, 1, 10, 100], # Regularization strength
    'solver': ['liblinear', 'saga'], # Solvers
    'penalty': ['l1', 'l2', 'elasticnet'], # Valid penalties
    'l1_ratio': [0.1, 0.5, 0.9] # Required for 'elasticnet', ignored otherwise
}

# Randomized Search CV for Logistic Regression
log_reg_random = RandomizedSearchCV(
    estimator=LogisticRegression(max_iter=1000), # max_iter increased for convergence
    param_distributions=param_distributions_log_reg,
    n_iter=20, # Number of parameter settings to sample
    cv=5, # Number of cross-validation folds
    scoring='accuracy',
    random_state=42, # For reproducibility
    n_jobs=-1 # Use all available cores
)

# Fit the model
log_reg_random.fit(X_train, y_train)

# Best parameters and score for Logistic Regression
print("Best Parameters for Logistic Regression:", log_reg_random.best_params_)
print("Best Accuracy for Logistic Regression:", log_reg_random.best_score_)

Best Parameters for Logistic Regression: {'solver': 'liblinear', 'penalty': 'l2', 'l1_ratio': 0.1, 'C': 10}
Best Accuracy for Logistic Regression: 0.8213059360905024
```

Best Accuracy: 0.8213 represents the highest accuracy achieved on the validation set during hyperparameter tuning.

Logistic Regression's performance improves with L2 regularization, and a higher C value helps it fit the data better. However, its accuracy (82.13%) is lower compared to the ensemble models, suggesting limited complexity handling.

7.1.2 Random Forest

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestClassifier

# Hyperparameter grid for Random Forest
param_distributions_rf = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

# Randomized Search CV for Random Forest
rf_random = RandomizedSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_distributions=param_distributions_rf,
    n_iter=50, # Number of parameter settings to sample
    cv=5, # Number of folds for cross-validation
    scoring='accuracy',
    random_state=42, # For reproducibility
    n_jobs=-1 # Use all available cores
)

# Fit the model
rf_random.fit(X_train, y_train)

# Best parameters and score for Random Forest
print("Best Parameters for Random Forest:", rf_random.best_params_)
print("Best Accuracy for Random Forest:", rf_random.best_score_)
```

```
Best Parameters for Random Forest: {'n_estimators': 300, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_depth': None, 'bootstrap': False}
Best Accuracy for Random Forest: 0.8845712934354018
```

Best Parameters

- `n_estimators`: 300: The number of decision trees in the forest. A higher number generally increases accuracy but also computational cost.
- `min_samples_split`: 2: Minimum samples required to split a node. A smaller value allows deeper splits, capturing more detail from the data.
- `min_samples_leaf`: 2: Minimum samples required to be at a leaf node. This prevents overly complex trees (reducing overfitting).

Best Accuracy: 0.8846 is the tuned Random Forest achieves high accuracy, indicating that these hyperparameters enable the model to generalize well.

The best parameters focus on controlling overfitting (via `min_samples_leaf` and `min_samples_split`) while maintaining model flexibility (with no maximum depth). Disabling bootstrapping allows each tree to learn from the entire dataset, which can sometimes improve performance.

7.1.3 Gradient Boosting Classifier

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import RandomizedSearchCV
import numpy as np

# Hyperparameter grid for Gradient Boosting
param_dist_gb = {
    'n_estimators': np.arange(50, 201, 10),
    'learning_rate': np.linspace(0.01, 0.3, 10),
    'max_depth': np.arange(2, 11),
    'min_samples_split': np.arange(2, 11),
    'min_samples_leaf': np.arange(1, 11),
    'subsample': np.linspace(0.5, 1.0, 6),
}

# Initialize Gradient Boosting Classifier
gb = GradientBoostingClassifier(random_state=42)

# RandomizedSearchCV for Gradient Boosting
random_search_gb = RandomizedSearchCV(
    estimator=gb,
    param_distributions=param_dist_gb,
    n_iter=20,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42
)

# Fit the model to the training data
random_search_gb.fit(X_train, y_train)

# Output the best parameters and best score
print("Best Parameters for Gradient Boosting:", random_search_gb.best_params_)
print("Best Accuracy for Gradient Boosting:", random_search_gb.best_score_)
```

```
Best Parameters for Gradient Boosting: {'subsample': 0.5, 'n_estimators': 180, 'min_samples_split': 5, 'min_samples_leaf': 4, 'max_depth': 8, 'learning_rate': 0.1388888888888889}
Best Accuracy for Gradient Boosting: 0.9024693087255601
```

Best Parameters

- **n_estimators:** 180: Number of boosting iterations. A smaller number prevents overfitting and reduces computational cost.
- **min_samples_split:** 5: Minimum samples required to split a node. A higher value smooths the decision boundary by preventing very deep splits.
- **learning_rate:** 0.1389: Determines the contribution of each tree to the final prediction. Lower values generally yield better results but require more estimators.

Best Accuracy: 0.9025 is the highest accuracy Gradient Boosting achieves among the three models, indicating that the combination of hyperparameters enables robust generalization.

The best parameters balance model complexity (**max_depth** and **min_samples_split**) with regularization (**subsample** and **learning_rate**). This balance is critical in boosting models to prevent overfitting while maintaining high performance.

7.2 Compare model performance before and after Tuning

		<i>Before</i>	<i>After</i>	<i>Change</i>
<i>Logistic Regression</i>	Accuracy	0.8285	0.8280	-0.0005
	Precision	0.8126	0.8289	+0.0163
	Recall	0.8538	0.8280	-0.0258
	F1 Score	0.8327	0.8279	-0.0048
<i>Random Forest Classifier</i>	Accuracy	0.8898	0.8922	+0.0024
	Precision	0.8900	0.8930	+0.0030
	Recall	0.8895	0.8922	+0.0027
	F1 Score	0.8897	0.8922	+0.0025
<i>Gradient Boosting Classifier</i>	Accuracy	0.8628	0.9067	+0.0439
	Precision	0.8447	0.9067	+0.0620
	Recall	0.8889	0.9067	+0.0178
	F1 Score	0.8663	0.9067	+0.0404

Logistic Regression

The performance remained largely the same or slightly decreased after tuning. This could indicate that Logistic Regression reached its optimal performance with default parameters or the data may not be well-suited for this model. Minimal impact from tuning suggests that Logistic Regression may not benefit significantly from hyperparameter adjustment in this case.

Effects of Parameter Changes

- The model showed negligible or slightly negative changes in metrics.
- Increasing C led to better precision but worsened recall, suggesting a trade-off in predicting positive cases accurately.
- Logistic Regression is not very sensitive to tuning and struggled to outperform its baseline performance.

Random Forest Classifier

The performance improved slightly in all metrics after tuning, indicating that the optimized hyperparameters (e.g., more estimators, adjusted split parameters) helped the model generalize better. Random Forest's improvement reflects the utility of hyperparameter tuning, but the changes are incremental due to its robust performance even with default parameters.

Effects of Parameter Changes

- Incremental improvements across all metrics demonstrate that Random Forest is robust even with default settings.
- Increasing the number of estimators enhanced stability and generalization.
- Adding constraints on splits (min_samples_leaf=2) slightly reduced overfitting.

Gradient Boosting Classifier

The most significant improvement was observed in Gradient Boosting, with all metrics increasing substantially. This shows that hyperparameter tuning (e.g., adjusting subsample, learning rate, number of estimators) greatly enhanced the model's ability to balance precision and recall. The consistent increase across metrics demonstrates Gradient Boosting's sensitivity to hyperparameter changes and its ability to leverage them for better performance.

Effects of Parameter Changes

- Significant improvements in all metrics show that Gradient Boosting is highly sensitive to hyperparameter tuning.
- Adding constraints (min_samples_split and min_samples_leaf) and subsampling improved generalization while maintaining high accuracy.
- Increasing tree depth and estimators captured complex relationships in the data.

Summary of Performance Improvements

<i>Model Accuracy</i>	<i>Change</i>	<i>Precision Change</i>	<i>Recall Change</i>	<i>F1-Score Change</i>
<i>Logistic Regression</i>	-0.0005	+0.0163	-0.0258	-0.0048
<i>Random Forest</i>	+0.0024	+0.0030	+0.0027	+0.0025
<i>Gradient Boosting</i>	+0.0439	+0.0620	+0.0178	+0.0404

- ✓ **Gradient Boosting** benefited the most from hyperparameter tuning, with a notable increase in all performance metrics. This shows it is the most adaptable model for this dataset.
- ✓ **Random Forest** showed moderate improvements, suggesting its performance is robust even with default parameters. It remains a reliable model with minimal tuning.
- ✓ **Logistic Regression** displayed little to no improvement after tuning, highlighting its simplicity but limited applicability for this task.

8. Comparison and Conclusion

8.1 Comparison

```
[ ] # Evaluate the performance of models after hyperparameter tuning using the best models

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Define the evaluation function
def evaluate_classification_model(model, X_test, y_test):
    """
    Evaluate a classification model on the test set and print performance metrics.

    Parameters:
    - model: The trained classification model.
    - X_test: Test features.
    - y_test: True labels for the test set.
    """
    # Predict the labels for the test set
    y_pred = model.predict(X_test)

    # Calculate performance metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average='weighted')
    recall = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')
    cm = confusion_matrix(y_test, y_pred)

    # Print metrics
    print(f"Model: {model.__class__.__name__}")
    print("Accuracy:", accuracy)
    print("Precision:", precision)
    print("Recall:", recall)
    print("F1 Score:", f1)
    print("Confusion Matrix:\n", cm)
    print("\n")

# Assuming the following variables exist from earlier in the code:
# - log_reg_random: RandomizedSearchCV object for Logistic Regression
# - rf_random: RandomizedSearchCV object for Random Forest
# - random_search_gb: RandomizedSearchCV object for Gradient Boosting
# - X_test: Test features
# - y_test: Test labels

# Evaluate Logistic Regression (after tuning)
evaluate_classification_model(log_reg_random.best_estimator_, X_test, y_test)

# Evaluate Random Forest (after tuning)
evaluate_classification_model(rf_random.best_estimator_, X_test, y_test)

# Evaluate Gradient Boosting (after tuning)
evaluate_classification_model(random_search_gb.best_estimator_, X_test, y_test)
```

Model: LogisticRegression
Accuracy: 0.8280850491185574
Precision: 0.8289083193629838
Recall: 0.8280850491185574
F1 Score: 0.827979589077778
Confusion Matrix:
[[5971 1463]
 [1092 6336]]

Model: RandomForestClassifier
Accuracy: 0.8922756022069708
Precision: 0.8930200156821391
Recall: 0.8922756022069708
F1 Score: 0.8922256426046243
Confusion Matrix:
[[6472 962]
 [639 6789]]

Model: GradientBoostingClassifier
Accuracy: 0.9067420266451353
Precision: 0.9067427125694981
Recall: 0.9067420266451353
F1 Score: 0.906741959090999
Confusion Matrix:
[[6746 688]
 [698 6730]]

Model	Strengths	Weaknesses
Logistic Regression	Interpretability: Easy to interpret and understand, suitable for simpler, transparent models.	Underfitting: Struggles with complex datasets and non-linear relationships, leading to underfitting.
	Fast Training: Computationally inexpensive and quick to train, even with large datasets.	Limited by Assumptions: Assumes linearity, making it less effective for non-linear data.
	Good for Linearly Separable Data: Works well with clear linear relationships between features and target.	Performance: Accuracy and other metrics are lower compared to ensemble models, making it less suitable for complex data.
Random Forest Classifier	Robustness: Performs well on noisy datasets and is less prone to overfitting than single decision trees.	Computational Complexity: Requires significant resources, especially with many trees (n_estimators).
	Handles Non-Linearity: Captures complex non-linear relationships.	- Interpretability: Harder to interpret compared to simpler models like Logistic Regression.
	Feature Importance: Offers insights into important features for prediction.	Memory Usage: Can be memory-intensive with a large number of trees and features.
Gradient Boosting Classifier	Good Generalization: After tuning, shows strong generalization with minimal drop-off from training to testing performance.	
	Best Performance: Achieves highest accuracy and precision, particularly after hyperparameter tuning.	Prone to Overfitting: Risk of overfitting without proper tuning, especially with many or deep trees.
	Handling Complex Patterns: Effectively captures complex patterns and non-linearity.	Computational Cost: Computationally expensive and slower to train due to the iterative nature of boosting.
	Handling Imbalanced Data: Performs well with imbalanced datasets, balancing precision and recall.	Complexity: Harder to interpret due to sequential tree-building, complicating understanding of decision-making processes.
	Tuning Flexibility: Highly flexible and can be fine-tuned to achieve superior results.	



```
Model: LogisticRegression
Accuracy: 0.8280850491185574
Precision: 0.8289083193629838
Recall: 0.8280850491185574
F1 Score: 0.827979589077778
Confusion Matrix:
[[5971 1463]
 [1092 6336]]
```



```
Model: RandomForestClassifier
Accuracy: 0.8922756022069708
Precision: 0.8930200156821391
Recall: 0.8922756022069708
F1 Score: 0.8922256426046243
Confusion Matrix:
[[6472 962]
 [ 639 6789]]
```



```
Model: GradientBoostingClassifier
Accuracy: 0.9067420266451353
Precision: 0.9067427125694981
Recall: 0.9067420266451353
F1 Score: 0.9067419590909999
Confusion Matrix:
[[6746 688]
 [ 698 6730]]
```

Based on these performance metrics generated from the 3 models,

Gradient Boosting Classifier emerged as the best model due to its **highest accuracy** and **balanced performance** across all metrics. After tuning, it outperformed both Logistic Regression and Random Forest, particularly in **precision** and **recall**.

8.2 Challenges encountered during the project and Solutions

Challenge	Description	Solutions
1. Data Quality Issues	Missing values ('?'), outliers, and imbalanced dataset affected performance.	Imputation for missing values (mean/median/mode). Detected and handled outliers using Z-score/IQR. Balanced classes using SMOTE or class weighting.
2. Feature Selection	Irrelevant/redundant features reduced model efficiency and could lead to overfitting.	Performed feature importance analysis using tree models/statistics. Applied PCA for dimensionality reduction.
3. Model Convergence	Logistic Regression or Gradient Boosting had slow convergence or failed to converge with improper initialization. Tree-based models overfit with defaults.	Used optimized solvers (liblinear, saga). Applied regularization (e.g., L2 for Logistic Regression). Tuning parameters like C, max_depth, subsample.
4. Computational Complexity	Random Forest and Gradient Boosting had high training time and memory usage. Grid Search was computationally expensive.	Used a smaller data subset for initial tuning. Leveraged parallel processing (n_jobs). Opted for Randomized Search over Grid Search.
5. Model Evaluation Challenges	Over-reliance on accuracy caused misleading results on imbalanced datasets. Interpreting AUC-ROC and business relevance was challenging.	Focused on F1-Score, Precision, and Recall. Used k-fold cross-validation for consistent evaluation. Visualized confusion matrices.

8.3 Conclusion Remarks

This report presented a comprehensive analysis and evaluation of machine learning models for predicting income levels based on demographic, educational, and employment characteristics. By addressing challenges such as missing values, outliers, and class imbalance, and applying robust preprocessing and feature engineering techniques, the models were effectively trained and optimized to solve the problem.

Key Insights on Model Effectiveness

The Gradient Boosting Classifier emerged as the top-performing model, achieving the highest accuracy (90.7%) and balanced precision, recall, and F1-score, proving its effectiveness in handling complex patterns and imbalanced data. The Random Forest model provided robust generalization and strong performance, making it a reliable alternative when computational efficiency is critical. Logistic Regression served as a baseline model, offering simplicity and interpretability but struggled with capturing non-linear relationships, limiting its effectiveness for this dataset.

Challenges Overcome

Data quality issues were resolved through imputation and outlier handling using log transformations and clipping, while class imbalance was mitigated with SMOTE, significantly improving minority class predictions. Computational costs during hyperparameter tuning were reduced by employing RandomizedSearchCV and limiting iterations for slower models.

Recommendations for Future Work

For high accuracy and handling complex data, Gradient Boosting should be the primary choice. Random Forest can be used in scenarios requiring interpretability and computational efficiency. Future work should explore advanced models like XGBoost or LightGBM, improve model interpretability with tools like SHAP and LIME, and optimize computational resources for scaling to larger datasets. This project highlights the effectiveness of machine learning in solving income prediction problems and sets a strong foundation for further refinement and application in similar classification tasks.

9. Documentation and Presentation

Data Set Link: <https://www.kaggle.com/datasets/wenruihu/adult-income-dataset/data>

Presentation Video: <https://drive.google.com/file/d/1L-nvIoBhhlUuHdob22nazcVTdWe5GBfe/view?usp=sharing>

Codebase:

- Python script: <https://drive.google.com/file/d/1KqrILHe-mPeKZy8gfhrN9-WQKd-tCCII/view?usp=sharing>
- PDF of the Notebook: <https://drive.google.com/file/d/12lmjMnst4-fFq1tIdY8hnTpAUxmPFFr9/view?usp=sharing>

Infrastructure Details: Hardware/software infrastructure used.

Device specifications	
Device name	DESKTOP-CHALANI
Processor	11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz 3.00 GHz
Installed RAM	12.0 GB (11.7 GB usable)
Device ID	2BDC5827-66FA-48D1-93EE-6FCF2671C227
Product ID	00327-36336-44522-AAOEM
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display

Device name	Imasha
Processor	12th Gen Intel(R) Core(TM) i7-12700H 2.30 GHz
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	BEA548CB-FF4C-4F91-B490-42EB3EAC26F5
Product ID	00331-10000-00001-AA114
System type	64-bit operating system, x64-based processor
Pen and touch	No pen or touch input is available for this display